

University of
HUDDERSFIELD
Inspiring global professionals

Module Code & Module Title

CMI3509 – Databases for Large Datasets

Academic Year

2025/26

Student Name: Heer Jadav

Student ID: 2967419

Submitted To: Dr. Alok Verma

Table of Contents

1	Introduction	2
2	Core Banking Transaction System Using PostgreSQL	3
2.1	Case Study Overview	3
2.2	Tutorial Involving PostgreSQL	4
3	E-Commerce Product Catalogue Using MongoDB	11
3.1	Case Study Overview	11
3.2	Tutorial Involving MongoDB	12
4	Professional Issues in Database Systems	16
5	Conclusion	17
6	References.....	18
7	Appendix	19

Abstract

Modern enterprise systems require database technologies that match the structure, scalability and operational requirements of their applications. This report builds two database tutorials on PostgreSQL and MongoDB to demonstrate the practical implementation of relational and NoSQL database paradigms. The first tutorial shows you how to create a core banking transaction system with PostgreSQL, focusing on transactional consistency, structured schemas, and SQL querying. The second tutorial shows an e-commerce product catalog using MongoDB. It focuses on flexible schemas, embedded documents and querying over documents. Finally, the report discusses professional issues related to data security, integrity, privacy and scalability in modern database systems. The report shows, through these tutorials, that the choice of a suitable database architecture depends on the nature of the data and the operational needs of the application domain.

1 Introduction

The rapid growth of data-intensive systems has drastically changed the database management practices across the industries. In a structured data environment where strict consistency and transactional integrity is required traditional relational database management systems (RDBMSs) still do well. However, contemporary applications are increasingly ingesting unstructured and semi-structured data at an enormous scale, which presents challenges to traditional relational architectures (Stonebraker, 2010).

Consequently, NoSQL databases are extensively adopted in applications that demand schema flexibility, horizontal scalability and high-performance distributed processing (Cattell, 2011). Instead of fully replacing relational databases, many organizations use a polyglot persistence strategy, where they choose different database technologies for the needs of specific applications (Sadalage & Fowler, 2012).

This report presents two practical database tutorials based on realistic enterprise use cases. The first tutorial demonstrates how to implement a core banking transaction system using PostgreSQL with strong ACID compliance and data consistency. The second tutorial demonstrates how to build an e-commerce product catalogue system using MongoDB that requires flexible document structures and scalable querying.

The report also delves into professional issues in database engineering including data privacy, security, transactional integrity, scalability, backup and recovery and the ethical responsibilities when handling sensitive user data.

2 Core Banking Transaction System Using PostgreSQL

2.1 Case Study Overview

Financial institutions need very reliable database systems to process millions of transactions securely and accurately. A core banking transaction system maintains customer records, account balances, deposits, withdrawals, and transfers of funds between accounts. These systems require strong transactional consistency because errors in financial processing can have serious operational and legal consequences.

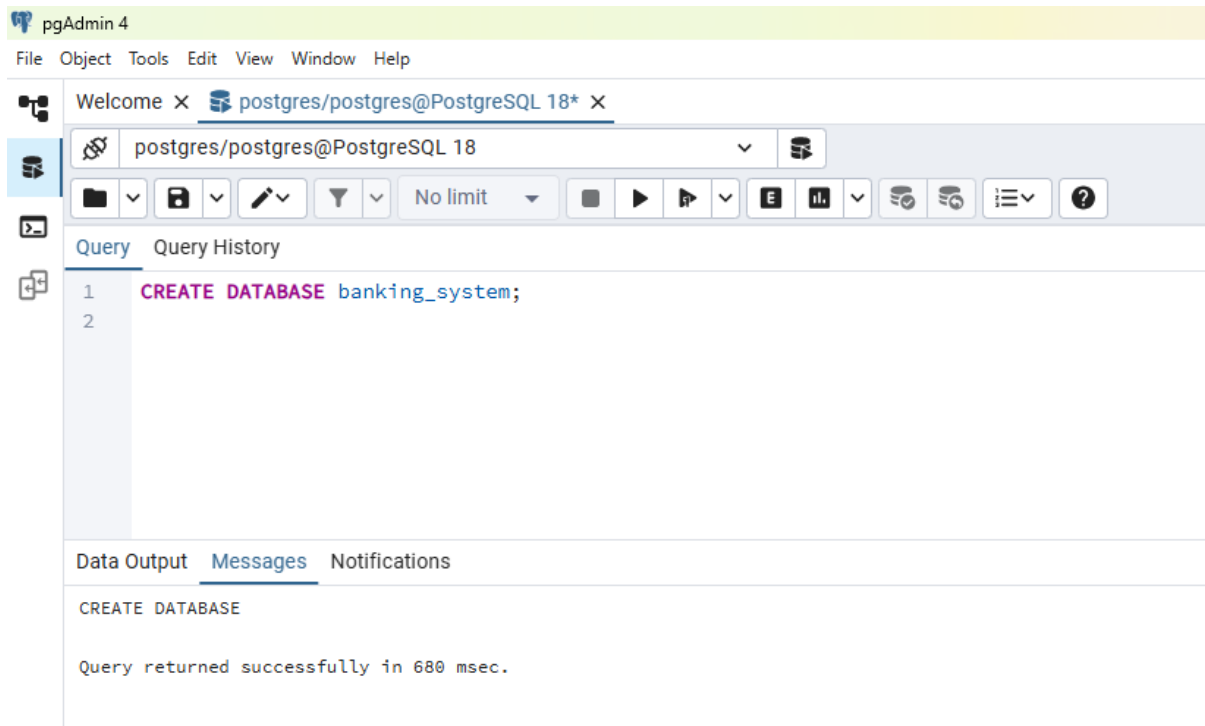
Banking systems data is very relational and structured. Entities like Customers, Accounts, Transactions, and Branches have strong relationships that require referential integrity and transactional reliability. PostgreSQL is very suitable for this use case due to full support for ACID properties, advanced concurrency control mechanisms and SQL-based relational modelling (Obe & Hsu, 2014).

In this tutorial, you will learn to create a simple banking database using PostgreSQL. It will include the creation of tables, relational modeling, insertion, querying, and transaction processing.

2.2 Tutorial Involving PostgreSQL

Step 1: Creating the Database

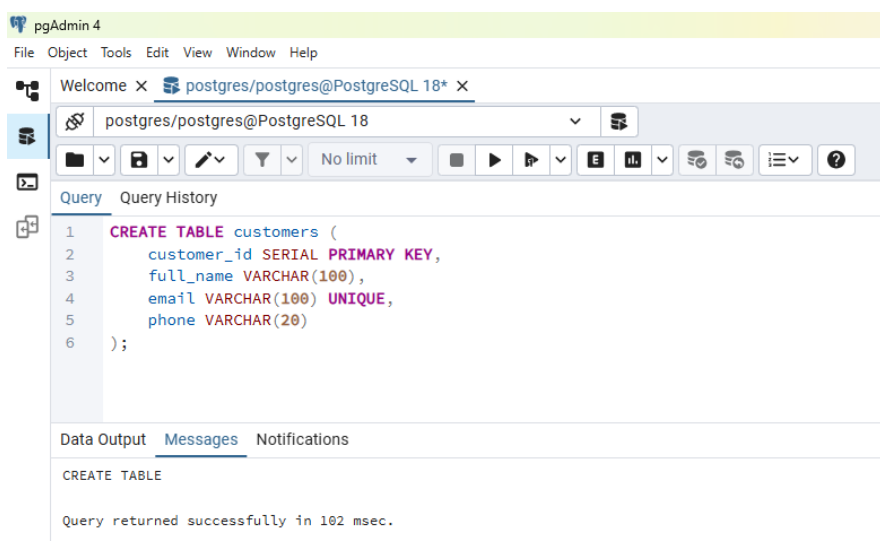
The first step is to create a new database named `banking_system`.



Step 2: Creating Tables

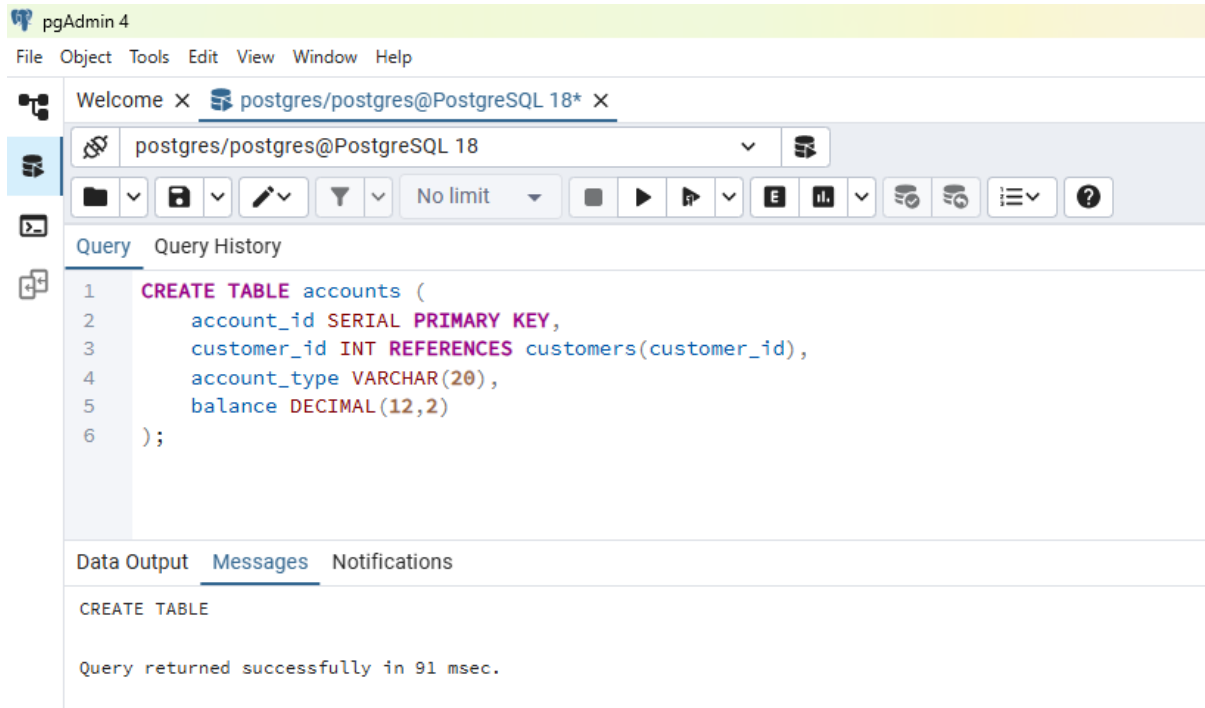
The banking system requires two primary tables: customers and accounts.

Creating the Customers Table



This table stores customer information. The `customer_id` field is automatically incremented using the SERIAL datatype, while the email field uses a unique constraint to prevent duplicate entries.

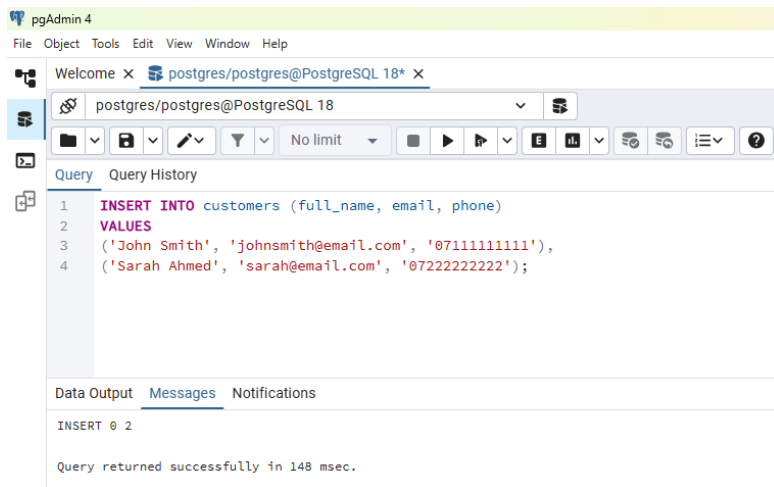
Creating the Accounts Table



The `customer_id` field acts as a foreign key, ensuring that every account belongs to a valid customer.

Step 3: Inserting Data

Insert Customer Records

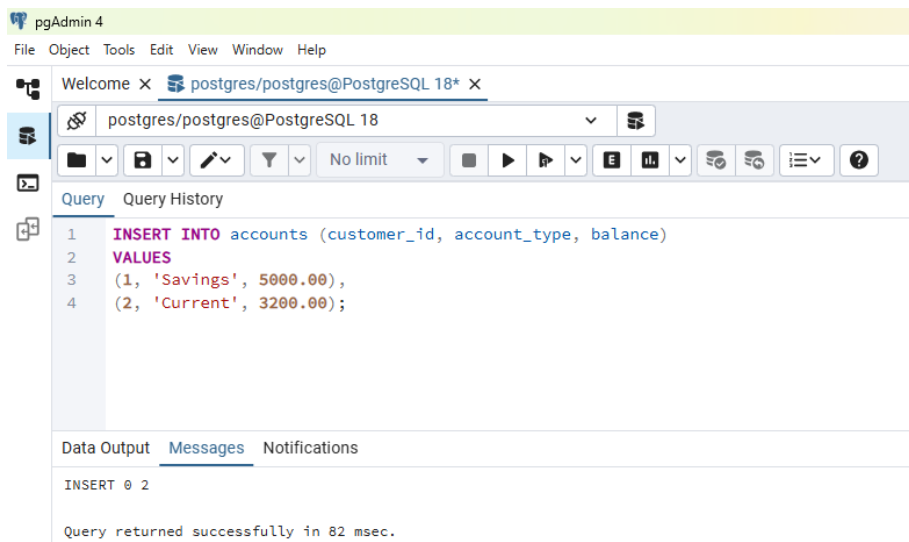


The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window displays a query editor for the 'postgres/postgres@PostgreSQL 18*' connection. The query is as follows:

```
1 INSERT INTO customers (full_name, email, phone)
2 VALUES
3 ('John Smith', 'johnsmith@email.com', '0711111111'),
4 ('Sarah Ahmed', 'sarah@email.com', '0722222222');
```

Below the query editor, the 'Messages' tab is active, showing the execution result: 'INSERT 0 2' and 'Query returned successfully in 148 msec.'

Insert Account Records



The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window displays a query editor for the 'postgres/postgres@PostgreSQL 18*' connection. The query is as follows:

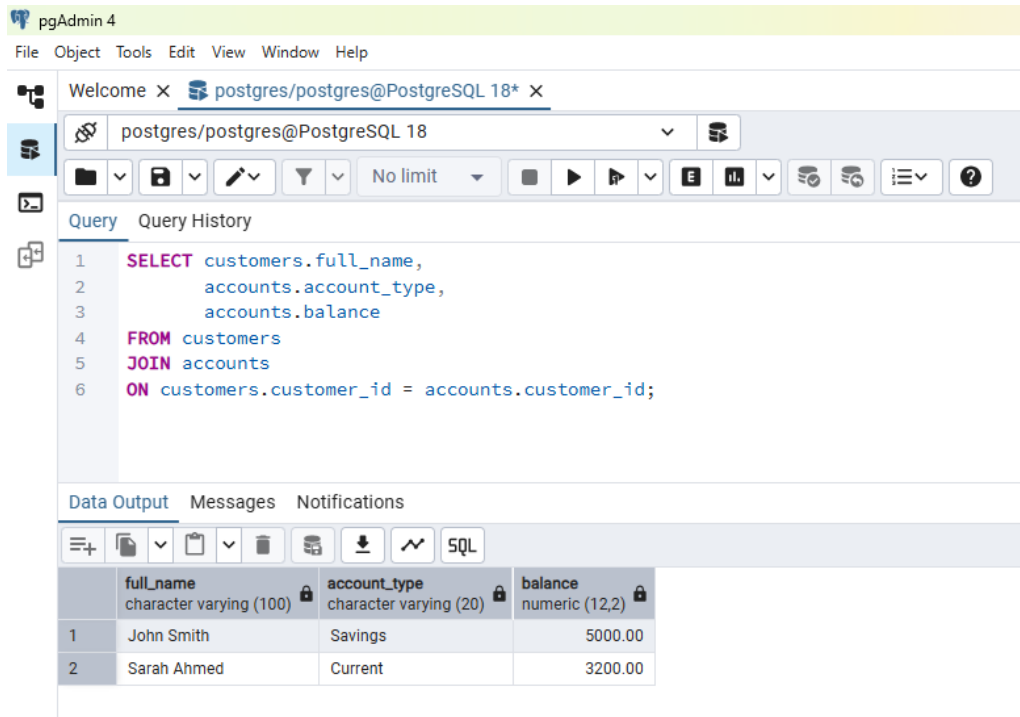
```
1 INSERT INTO accounts (customer_id, account_type, balance)
2 VALUES
3 (1, 'Savings', 5000.00),
4 (2, 'Current', 3200.00);
```

Below the query editor, the 'Messages' tab is active, showing the execution result: 'INSERT 0 2' and 'Query returned successfully in 82 msec.'

These commands populate the database with sample customer and account data.

Step 4: Querying the Database

To retrieve all customer account information:



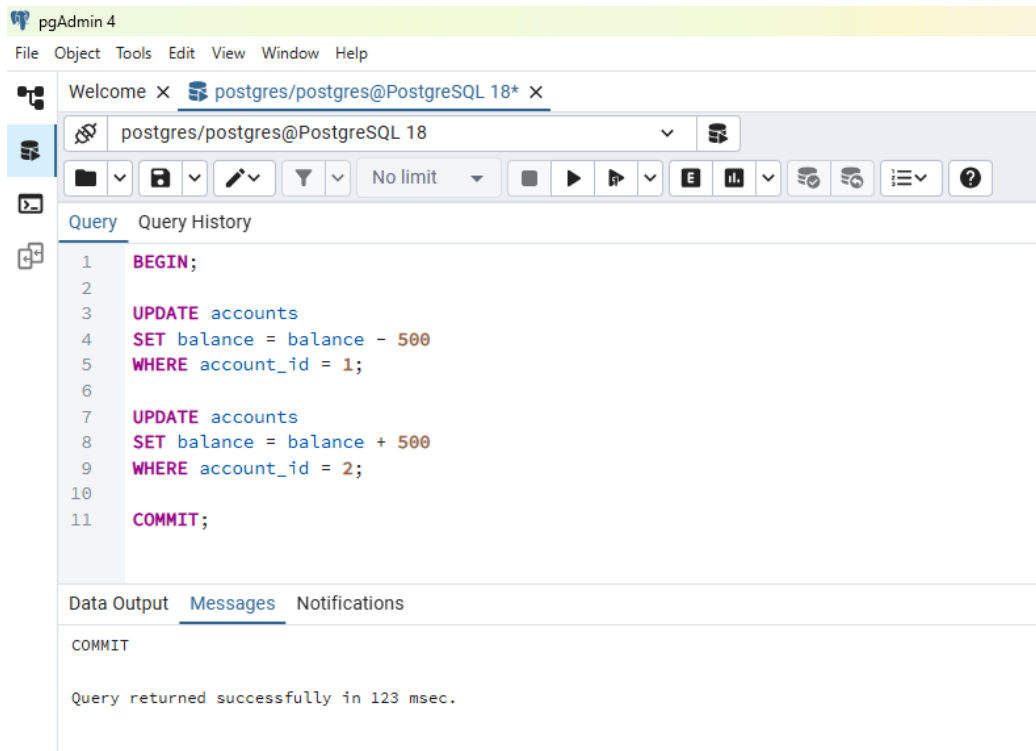
The screenshot shows the pgAdmin 4 web interface. The top bar indicates the user is logged in as 'postgres/postgres@PostgreSQL 18'. The main panel is divided into two sections: 'Query' and 'Query History'. The 'Query' section contains a SQL query that joins the 'customers' and 'accounts' tables to retrieve customer information and account balances. The 'Query History' section is empty. Below the query, the 'Data Output' tab is active, displaying the results of the query in a table format. The table has three columns: 'full_name', 'account_type', and 'balance'. The results show two rows: one for 'John Smith' with a 'Savings' account and a balance of 5000.00, and another for 'Sarah Ahmed' with a 'Current' account and a balance of 3200.00.

```
1 SELECT customers.full_name,  
2         accounts.account_type,  
3         accounts.balance  
4 FROM customers  
5 JOIN accounts  
6 ON customers.customer_id = accounts.customer_id;
```

	full_name character varying (100)	account_type character varying (20)	balance numeric (12,2)
1	John Smith	Savings	5000.00
2	Sarah Ahmed	Current	3200.00

Step 5: Performing a Transaction

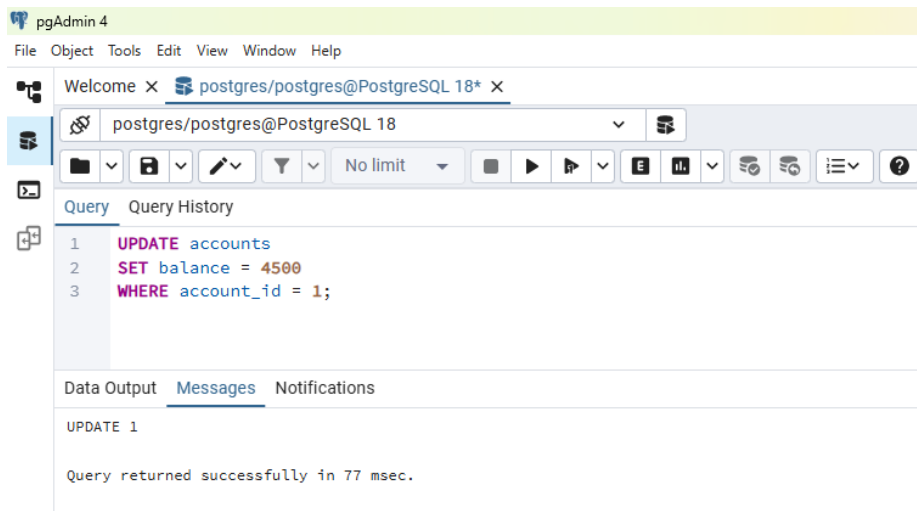
One of PostgreSQL's major strengths is transactional consistency. The following example transfers £500 from one account to another.



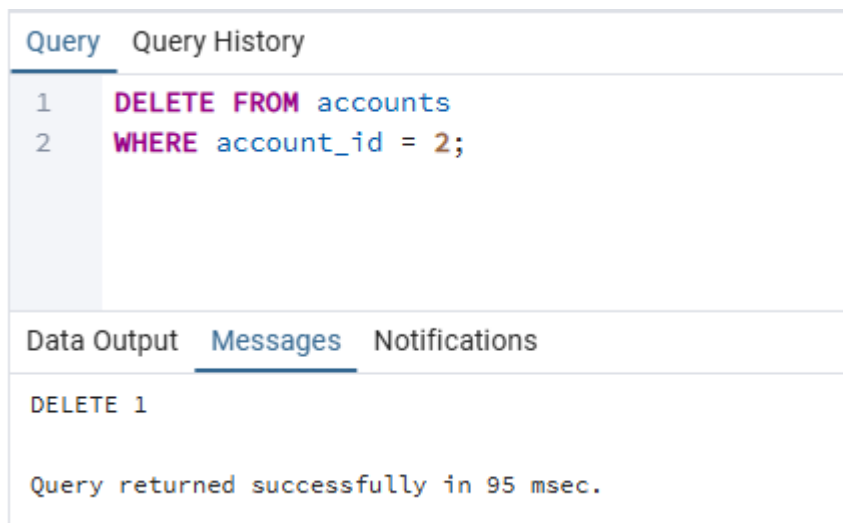
The BEGIN and COMMIT commands ensure that both updates succeed together. If any error occurs during execution, the transaction can be cancelled using ROLLBACK, preventing partial updates.

Step 6: Updating and Deleting Records

Updating an Account Balance



Deleting an Account



These operations demonstrate PostgreSQL's support for standard CRUD (Create, Read, Update, Delete) functionality.

Discussion of PostgreSQL Strengths

PostgreSQL is a good choice for banking systems because it prioritizes reliability, consistency, and transactional safety. It uses Multi-Version Concurrency Control (MVCC) which allows multiple users to access the database at the same time without significant locking conflicts (Obe & Hsu, 2014).

PostgreSQL also supports advanced indexing, stored procedures, views and backup mechanisms which makes it suitable for enterprise scale financial systems. The structured relational model provides strong integrity constraints. It helps to avoid inconsistent or invalid financial data.

However, relational databases may be less efficient for highly dynamic or semi-structured datasets with frequent changes in schema structures.

3 E-Commerce Product Catalogue Using MongoDB

3.1 Case Study Overview

Modern e-commerce systems deal with a large scale of very diverse product data. The attributes found in products can vary based on the category. For example, a smartphone might include technical specifications such as RAM, and processor type, while clothing products might include color, size, and material information.

This level of schema variability is not well suited to traditional relational databases, which require fixed table structures. To address this issue, MongoDB uses a NoSQL document-oriented approach that stores data in flexible BSON documents (Chodorow, 2013).

MongoDB is also a great choice for e-commerce platforms as it comes with horizontal scalability, flexible schemas, embedded documents, and fast document retrieval. This tutorial shows how MongoDB can be used to implement a simplified product catalogue system.

3.2 Tutorial Involving MongoDB

Step 1: Creating the Database

Open the MongoDB shell and create a database named `ecommerce_catalogue`.

```
test> use ecommerce_catalogue
switched to db ecommerce_catalogue
```

MongoDB automatically creates the database when data is inserted.

Step 2: Creating a Collection

MongoDB stores data in collections instead of relational tables.

Create a collection named `products`:

```
ecommerce_catalogue> db.createCollection("products")
{ ok: 1 }
```

Step 3: Inserting Documents

Insert a product document into the collection:

```
ecommerce_catalogue> db.products.insertOne({
  name: "Gaming Laptop",
  category: "Electronics",
  price: 1200,
  attributes: {
    RAM: "16GB",
    CPU: "Intel i7",
    Storage: "1TB SSD"
  },
  reviews: [
    {
      user: "John",
      rating: 5
    },
    {
      user: "Sarah",
      rating: 4
    }
  ]
})
{
  acknowledged: true,
  insertedId: ObjectId('69fc6be7603d7244e17c2907')
}
```

This document demonstrates MongoDB's flexible schema structure. Product attributes and reviews are embedded directly inside the document.

Step 4: Querying Documents

Retrieve All Products

```
ecommerce_catalogue> db.products.find()
[
  {
    _id: ObjectId('69fc6be7603d7244e17c2907'),
    name: 'Gaming Laptop',
    category: 'Electronics',
    price: 1200,
    attributes: { RAM: '16GB', CPU: 'Intel i7', Storage: '1TB SSD' },
    reviews: [ { user: 'John', rating: 5 }, { user: 'Sarah', rating: 4 } ]
  }
]
```

Retrieve Electronics Products Only

```
ecommerce_catalogue> db.products.find({ category: "Electronics" })
[
  {
    _id: ObjectId('69fc6be7603d7244e17c2907'),
    name: 'Gaming Laptop',
    category: 'Electronics',
    price: 1200,
    attributes: { RAM: '16GB', CPU: 'Intel i7', Storage: '1TB SSD' },
    reviews: [ { user: 'John', rating: 5 }, { user: 'Sarah', rating: 4 } ]
  }
]
```

Retrieve Products with Price Greater Than 1000

```
ecommerce_catalogue> db.products.find({ price: { $gt: 1000 } })
[
  {
    _id: ObjectId('69fc6be7603d7244e17c2907'),
    name: 'Gaming Laptop',
    category: 'Electronics',
    price: 1200,
    attributes: { RAM: '16GB', CPU: 'Intel i7', Storage: '1TB SSD' },
    reviews: [ { user: 'John', rating: 5 }, { user: 'Sarah', rating: 4 } ]
  }
]
```

These queries demonstrate MongoDB's JSON-style querying syntax.

Step 5: Updating Documents

The following query updates the product price:

```
ecommerce_catalogue> db.products.updateOne(
|   { name: "Gaming Laptop" },
|   { $set: { price: 1350 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

MongoDB allows partial updates without modifying the entire document.

Step 6: Adding Additional Reviews

```
ecommerce_catalogue> db.products.updateOne(
|   { name: "Gaming Laptop" },
|   {
|     $push: {
|       reviews: {
|         user: "David",
|         rating: 5
|       }
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

The \$push operator inserts a new review into the embedded review array.

Step 7: Deleting Documents

```
ecommerce_catalogue> db.products.deleteOne({ name: "Gaming Laptop" })  
{ acknowledged: true, deletedCount: 1 }  
ecommerce_catalogue>
```

This command removes the selected product document from the collection.

Discussion of MongoDB Strengths

MongoDB offers considerable benefits to applications featuring semi-structured or rapidly changing data. Its schema is flexible, which allows developers to modify the data models of their application without complicated schema migrations.

Embedded documents provide better performance as associated information can be fetched in a single read operation. MongoDB also supports horizontal scaling through sharding and is very effective in large scale distributed systems (Moniruzzaman & Hossain, 2013).

However, in highly sensitive financial environments, MongoDB may not offer the same level of strict transactional consistency as relational systems. Therefore, it is more suitable for applications where scalability and flexibility are more important than strict transactional guarantees.

4 Professional Issues in Database Systems

Modern database systems store vast amounts of sensitive personal and organizational data . Therefore, database engineers and organizations need to consider a number of professional and ethical responsibilities in the design and deployment of database solutions.

Data breaches can expose sensitive personal, financial and commercial data. To protect the stored data organizations, we need to deploy strong authentication mechanisms, encryption, role-based access control and periodic security auditing. Personal data must also be handled in accordance with legislation such as the UK GDPR.

Data integrity has a vital role to play in systems such as banking applications where incorrect or inconsistent data can have serious financial consequences. PostgreSQL and other relational databases guarantee strong consistency via ACID transactions.

Enterprise systems today must be available under heavy traffic loads. NoSQL databases (MongoDB, for example) can scale better with distributed architectures and horizontal scaling. But engineers must balance scalability with consistency and reliability.

“Organizations need to ensure their data collection and storage practices are transparent and ethical. User data should only be collected when needed, stored securely and managed responsibly to prevent misuse or improper access.

5 Conclusion

Two useful database tutorials using PostgreSQL and MongoDB were developed in this report to show the implementation of relational and NoSQL database technologies into realistic enterprise environments.

In the PostgreSQL tutorial we saw how relational databases give you powerful transactional consistency, structured schemas, and reliable ACID-compliant processing for financial systems. The MongoDB tutorial showcased the schema flexibility, embedded data structures, and scalable querying capabilities of document-oriented NoSQL databases for modern e-commerce platforms.

The report also noted that there is no single database technology that is best for all uses. The database selection is instead based on matching the selected DBMS with the structure, scalability and operational requirements of the application domain. Currently, companies tend to use polyglot persistence strategies in organizations by combining several database technologies to achieve the best system performance.

Finally, the report highlighted the significance of professional issues in database engineering, such as data privacy, security, scalability, and ethical responsibility. Thus, effective database design is a combination of technical skills and responsible management of the data.

6 References

Cattell, R. (2011). Scalable SQL and NoSQL data stores. *Acm Sigmod Record*, 39(4), 12-27.

Chodorow, K. (2013). *MongoDB: the definitive guide*. " O'Reilly Media, Inc."

Moniruzzaman, A. B. M., & Hossain, S. A. (2013). Nosql database: New era of databases for big data analytics-classification, characteristics and comparison. *arXiv preprint arXiv:1307.0191*.

Obe, R. O., & Hsu, L. S. (2017). *PostgreSQL: up and running: a practical guide to the advanced open source database*. " O'Reilly Media, Inc."

Sadalage, P. J., & Fowler, M. (2013). *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education.

Stonebraker, M. (2010). SQL databases v. NoSQL databases. *Communications of the ACM*, 53(4), 10–11. <https://doi.org/10.1145/1721654.1721659>

7 Appendix

PostgreSQL Queries:

```
CREATE DATABASE banking_system;
```

```
\c banking_system;
```

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    full_name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    phone VARCHAR(20)  
);
```

```
CREATE TABLE accounts (  
    account_id SERIAL PRIMARY KEY,  
    customer_id INT REFERENCES customers(customer_id),  
    account_type VARCHAR(20),  
    balance DECIMAL(12,2)  
);
```

```
INSERT INTO customers (full_name, email, phone)  
VALUES  
('John Smith', 'johnsmith@email.com', '0711111111'),  
('Sarah Ahmed', 'sarah@email.com', '0722222222');
```

```
INSERT INTO accounts (customer_id, account_type, balance)
```

```
VALUES
```

```
(1, 'Savings', 5000.00),
```

```
(2, 'Current', 3200.00);
```

```
SELECT customers.full_name,
```

```
       accounts.account_type,
```

```
       accounts.balance
```

```
FROM customers
```

```
JOIN accounts
```

```
ON customers.customer_id = accounts.customer_id;
```

```
BEGIN;
```

```
UPDATE accounts
```

```
SET balance = balance - 500
```

```
WHERE account_id = 1;
```

```
UPDATE accounts
```

```
SET balance = balance + 500
```

```
WHERE account_id = 2;
```

```
COMMIT;
```

```
UPDATE accounts
```

```
SET balance = 4500
```

```
WHERE account_id = 1;
```

```
DELETE FROM accounts
```

```
WHERE account_id = 2;
```

MongoDB Queries:

```
use ecommerce_catalogue
```

```
db.createCollection("products")
```

```
db.products.insertOne({  
  name: "Gaming Laptop",  
  category: "Electronics",  
  price: 1200,  
  attributes: {  
    RAM: "16GB",  
    CPU: "Intel i7",  
    Storage: "1TB SSD"  
  },  
  reviews: [  
    { user: "John", rating: 5 },  
    { user: "Sarah", rating: 4 }  
  ]  
})
```

```
// Retrieve all products
```

```
db.products.find()
```



```
// Retrieve electronics products

db.products.find({ category: "Electronics" })


// Retrieve products with price greater than 1000

db.products.find({ price: { $gt: 1000 } })


// Update price

db.products.updateOne(
  { name: "Gaming Laptop" },
  { $set: { price: 1350 } }
)
```

```
// Add new review

db.products.updateOne(
  { name: "Gaming Laptop" },
  {
    $push: {
      reviews: {
        user: "David",
        rating: 5
      }
    }
  }
)

db.products.deleteOne({ name: "Gaming Laptop" })
```